Project Report: Fine-Grained Role and Attribute-Based Access Control

1. Executive Summary

This project involved architecting and deploying a secure, scalable, and granular access control system using AWS native services. The objective was to implement a mechanism where user identity is verified through Cognito, mapped to specific IAM roles based on group membership, and validated by API Gateway at the edge.

2. Architectural Components

The system leverages the following core components:

- **Amazon Cognito:** Manages user authentication and directory services, utilizing User Pool Groups to categorize users (Admin, Dev, Ops).
- **Identity Pools:** Facilitates the exchange of JWTs (IdTokens) for temporary AWS IAM credentials via STS (Security Token Service).
- **API Gateway:** Acts as the IAM Authorizer ("The Bouncer"), enforcing access control by validating that the temporary credentials possess the required IAM permissions for the requested route.
- **Pre-Token Generation Lambda:** Intercepts the authentication flow to inject custom claims (e.g., `department`, `tenant_id`) into the JWT, enabling Attribute-Based Access Control (ABAC).
- **Backend Lambda:** Executes business logic only after authorization is verified at the API Gateway level.

3. The Security Flow (The "Identity Chain")

The verification journey follows this established flow:

1. **Authentication:** The user authenticates with Cognito and receives a JWT.
2. **Federation:** The user exchanges the JWT for temporary credentials from the Identity Pool.
3. **Authorization (RBAC):** The API Gateway performs an `AWS_IAM` authorization check. If the IAM role associated with the credentials lacks the path-specific permission, access is denied at the edge (403 Forbidden).
4. **Attribute Injection (ABAC):** The Pre-Token Lambda injects departmental attributes into the token payload, allowing for data-level filtering inside the application layer.

4. Key Observations & Validation Results

- **Edge-Level Security:** We successfully demonstrated that unauthorized access attempts are blocked by API Gateway, preventing unauthorized requests from ever triggering the backend Lambda (optimizing cost and security).
- **Fine-Grained RBAC:** Verification confirmed that `dev_user` successfully accessed `/dev/data` but was restricted from `/admin/settings`.
- **ABAC Readiness:** By integrating the Pre-Token Generation trigger, the architecture is now primed for advanced resource-level filtering, allowing for multi-tenant data isolation based on custom claims.

5. Conclusion

This architecture successfully separates authentication from authorization and moves the security boundary to the API edge. The system is production-ready, highly modular, and minimizes backend compute costs by enforcing authorization before business logic execution.